Pattern Detection Using Convolutional Neural Network (CNN)

Saad Rahman
Computer Science and Electrical Engineering

University of Maryland Baltimore County
Baltimore, US
rahman2@umbc.edu

***Abstract* — In this project, a method of detecting patterns in images using Convolutional Neural Network (CNN) is demonstrated. The number and the position of the patterns are found and the method is implemented in Field Programmable Gate Array (FPGA).**

Keywords—CNN, Sobel filter, convolution, maxpooling, kernel, FPGA (*key words****)***

I. INTRODUCTION

Convolution is a mathematical operation on two functions to produce a third function, giving the integral of the pointwise multiplication of the two functions as a function of the amount that one of the original functions is translated. Convolution is a ubiquitous operation in signal and image processing. In this project, two images were taken [1]. One for pattern and denoted as 'pattern image' and one for the sample called 'sample image'. After that, both images are converted into grayscale and convoluted with an edge detecting filter. Then after edge detection the pattern and sample image is convoluted again and the point with high convolution value is targeted as the pattern. The entire process is designed in Verilog.

II. PHASES OF THE PROJECT

A. *First phase*

The objective of the first phase is to design the software prototype of the project using MATLAB.As an edge detection feature extraction the Sobel filter is used because of its simplicity and low computational cost. Firstly, the pattern and sample image is resized and then Sobel filter is applied on both the resized pattern image and sample images for feature extraction and edge detection. After that, both filtered images are convoluted again with each other. Then, the maximum value of the convolution is computed and a threshold value is set, each value of the convolution is compared to threshold value. Any value greater than the threshold value is considered of as higher match between the two images and replacing these values with 0 and other values as 1 which produces a final output image with black dots in the positions of detected patterns. This process is called maxpooling. As the number of bits required to show the pixel values and the size of images can be found in software verification. So, implementing the project in software helps to optimize the memory allocation for hardware implementation.

B. *Second phase*

The second phase of this project is aimed at design and verification of the convolution and maxpooling kernels. Three modules are designed to handle these tasks. One for the convolution of the pattern image with the filter, one for the convolution of the sample image with the filter and the last one is for the convolution of the pattern and sample image. Every module output was verified using separate testbench and the previously obtained data from phase one. The convolution was computed using a counter to pick proper pixel values from the sample and pattern images and multiplying them with the values of Sobel filter. For the maxpooling of the final convoluted image, no separate module is needed. The pixel values of Final convoluted image was sent to a comparator in the same clock edge of storing the final convoluted image in a memory. Then, with the help of the comparator,the maximum value of the final convoluted image is determined. The threshold value is set as the maximum value multiplied by 0.8.

The number of cycles taken for each process to finish, the parallel and serial operations taken in the processes have also been described in later stage. Two counters in parallel are used to pick the pixel values from the convoluted and pattern images and storing them in two different memories. Moreover, four counters are also used in parallel for picking the pixel values from the convoluted pattern and sample images.

C. *Third phase*

The phase three of the project is intended to complete all the kernel design and the state machine design. The implemented state machine has five states and those steps are controlled by control signals. A threshold value is set for detecting the pattern. After comparing the output of the final convoluted image with the threshold, pattern is detected and 0 is stored in a new memory where the value of final convoluted image exceeds the threshold and otherwise 1 is stored.

D. *Fourth phase*

The goal of phase four is to design a full working system. The software prototype of the first phase is to be implemented on hardware. The number of detected pattern should be shown in the monitor via VGA. Two counters are used for interfacing with the VGA and to provide row and column of a two-dimensional greyscale image from an one dimensional array. Also, a clock divider module was used to convert the FPGA operating frequency into 25 MHz which is the native frequency of VGA port.

III. IMPLEMENTATION PROCEDURE

The design of the prototype was implemented both in software and hardware. The details of the software and hardware implementation of the design are described here.

A. *Software implementation*

As an edge detection filter, Sobel filter [2] is used. A typical Sobel filter is shown below in figure 1:

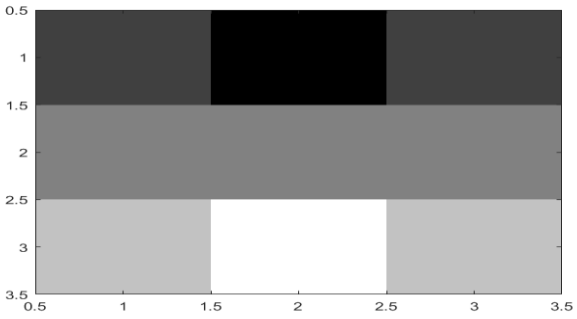

Fig. 1. Chosen Sobel filter for pattern detection

Process tree of the software prototype with the intermediate steps is shown in figure 2.

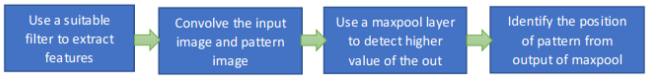

Fig. 2. Process tree of the software prototype

To optimize the entire process, the sample and pattern image are converted into grayscale and resized. The number of bits and memory required in every step of the project is tabled below in table x:

TABLE I. BITS AND MEMORY REQUIRED

Layer	Memory used (Bits)	Bits used
B&W Pattern Image	30 × 27 × 8	8
B&W Sample Image	440 × 340 × 8	8
Filter	3 × 3	8
Convoluted Pattern Image	28 × 25 × 10	10
Convoluted Sample Image	438 × 338 × 10	10
Final Image	411 × 314 × 26	26
Maxpooled Image	205 × 157 × 26	26
Output	205 × 157	1

B. *Hardware implementation*

The architecture of the hardware implemented project with all the steps involving convolution, maxpooling and detecting pattern is addressed in figure 4:

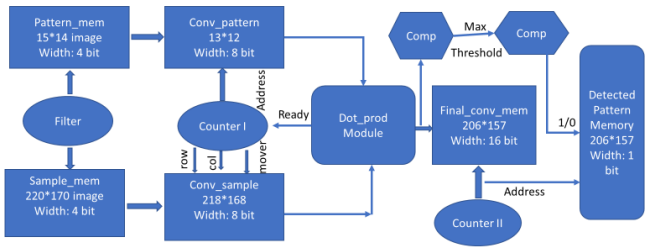

Fig. 3. Hardware architecture of the design

- The convolution of pattern and sample with Sobel filter is done and then the convoluted pattern and image is convoluted again.
- Comparators and counters are used to pull the values of the final convoluted image and comparing with threshold and provide decisions about the pattern detection.
- The state machine designed for this project has five steps. Each of the steps performs a specified task. If completed, then the next step is selected to perform the next set of tasks.
- The steps of the state machine are controlled by some control signals. The control signals associated with the steps and their description is provided in table 2.

TABLE II. CONTROL SIGNALS USED

Control Signal	Step	Description
convp_done	1	Goes high when convolution of pattern image is completed
convs_done	2	Goes high when convolution of sample image is completed
convfinal_done	3	Goes high when convolution of pattern image and sample image is completed
pat_detect_done	4	Goes high when maxpooling and comparison is completed
done	5	Pattern detection and storing output data into memory is completed

IV. RESULTS AND DISCUSSION

A. *Software prototype*

For software implementation, three types of filters were available for use. They are:

- Sobel filter (Vertical edge)
- Sobel filter (Horizontal edge)
- Laplasian filter.

The Sobel filter is used for edge detection because of its computational advantage and ease of use. Laplasian filter is also great option. The comparison of edge detection is depicted in figure 4.

Fig. 4. Edge detection by different filters

The software prototype of the project, coded in MATLAB performs well and can detect patterns successfully using different pattern images. Every time, the location of detected pattern was replaced by black dots. The results are shown in figure 5:

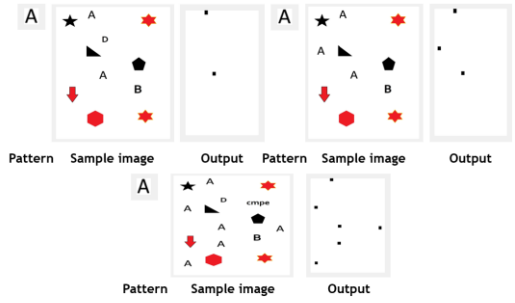

Fig. 5. Detected pattern using different sample images

B. *Hardware prototype*

The hardware implementation results are separated into x parts and described separately below.

A. *Cycle requirement*

The number of cycles taken to perform the immediate steps of whole operation are presented in table x.

TABLE III. CYCLES TAKEN IN EACH STEP

Operations	Cycle taken
Reading stored pattern image	178
Reading stored sample image	36957
Storing convoluted pattern image	155
Storing convoluted sample image	36623
Doing convolution of pattern and sample image	32342
Storing final convoluted image in memory	32342

B. *Implementation analysis*

After the completion of the design in Verilog, the timing, power and utilization analysis can be determined. Tools used in synthetization and implementation is Vivado 2018.1 and ISE Xilinx 14.7. The RTL level implementation of the design, timing analysis, resource utilization and power analysis are presented in figure 6 , 7, 8, 9 respectively.

The clock used in the project is 40 ns as the worst pulse width slack is 18.950 ns.

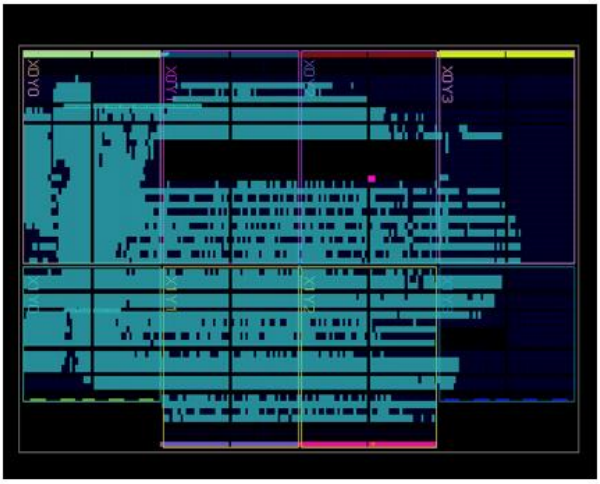

Fig. 6. RTL level design in Vivado

Fig. 7. Timing analysis

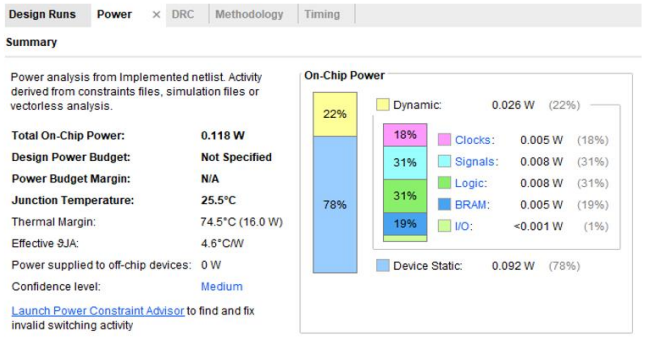

Fig. 8. Power analysis

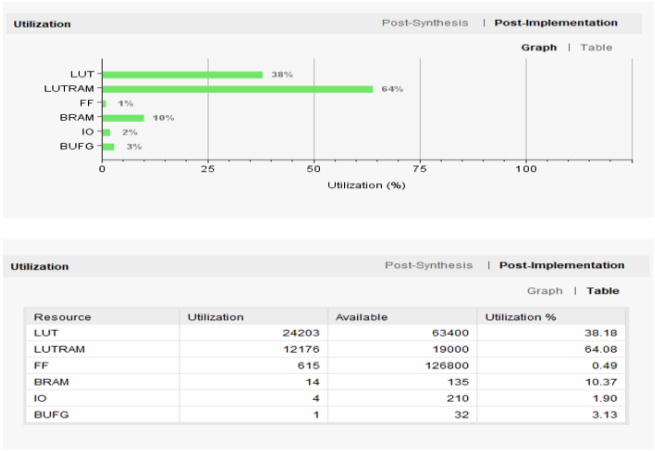

Fig. 9. Utilization analysis of the design

C. *Design verification*

After simulating the design with testbench the simulation result is depicted in figure 10.

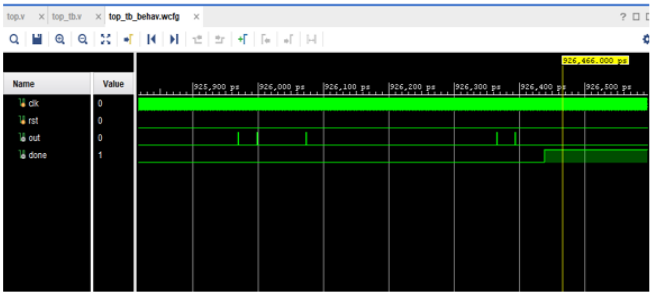

Fig. 10. Output of hardware simulation

Signal ‘out’ is set to 1 whenever a pattern is detected. From simulation it’s evident that the design can detect pattern successfully.

V. Challenges

A major challenge faced is to show the detected pattern in the monitor via VGA. The locations of the detected pattern are correct still no solution was found to show them on monitor using VGA. But, the output data from the memory was written in a test file and then using Python the data was reconstructed in an image. The output image on the lest shows the detected pattern and output in the right is VGA output which is depicted in figure 11.

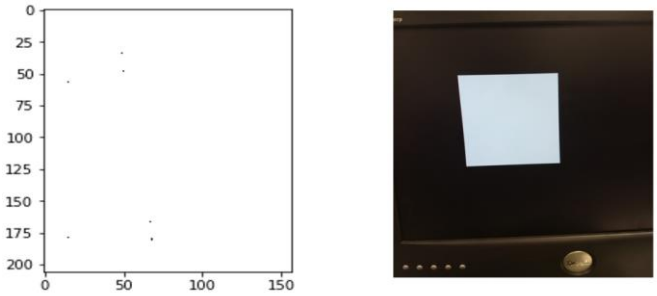

Fig. 11. VGA ouput

VI. Conclusion

A functional FPGA based pattern detector using CNN is designed in the project. Pattern detection is fully functional in simulation. The memory, power, resources, clock speed required to design the project are obtained and presented. The project is designed in both software and hardware and their verification is described separately.

Acknowledgment

I am grateful to the course instructor and the grader for constant helps and suggestions. I am also thankful to University of Maryland Baltimore county for providing the Artix-7 FPGA board.

References

[1] https://en.wikipedia.org/wiki/Convolution
[2] https://en.wikipedia.org/wiki/Sobel_operator

[3] https://www.mathworks.com/discovery/pattern-recognition.html